Ejercicio 1

Crea una clase sencilla llamada Persona que represente una entidad básica.

Debe incluir un constructor primario con un atributo inmutable nombre: String y un atributo mutable edad: Int.

Define un método llamado cumplirAnios() que incremente la edad en 1 y muestre un mensaje de felicitación por pantalla.

Ejercicio 2

Define una clase llamada Automovil que use únicamente el constructor primario para recibir marca: String y anio: Int? con un valor por defecto de 2026. Agrega una propiedad mutable encendido: Boolean inicializada en false y un método arrancar().

Ejercicio 3

Crea una class genérica llamada MascotaClass(val nombre: String, val edad: Int) y una data class llamada MascotaData(val nombre: String, val edad: Int).

Instancia un objeto de cada una e imprímelos usando println()

Ejercicio 4

Crea una clase llamada ConfiguracionSistema cuyo constructor primario sea privado (private constructor) y reciba un parámetro val clave: String.

Añade un constructor secundario público que no reciba parámetros y que invoque al constructor primario utilizando el valor predeterminado "CLAVE_DEFECTO".

Explica qué pasa si intentas llamar directamente al constructor primario desde la función main()

Ejercicio 5

Crea una clase llamada Curso que reciba nombre: String y cupoMaximo: Int en su constructor primario.

Añade un primer bloque init que utilice la función require() para asegurar que el nombre no esté en blanco (!nombre.isBlank()).

Añade un segundo bloque init que verifique mediante require() que el cupoMaximo sea mayor o igual a 5.

Demuestra qué ocurre al intentar crear un objeto con un cupo inválido (ej. 3).

Ejercicio 6

Define dos tipos de clases diferentes con las mismas propiedades (val id: Int, val titulo: String):

Una clase estándar LibroClass.

Una clase de datos LibroDataClass.

Escribe la función main() para demostrar el comportamiento que genera el compilador:

- Instancia dos objetos de LibroClass con exactamente los mismos datos y compáralos con el operador ==. Explica en consola qué imprime y por qué.
- Instancia dos objetos de LibroDataClass con los mismos datos y compáralos con ==. Explica la diferencia.

Ejercicio 7

Crea una data class llamada Pedido con las propiedades id: Int, producto: String y estado: String. Escribe un programa (fun main()) que haga lo siguiente:

Instancie un pedido inicial con id = 101, producto = "Portátil", y estado = "Pendiente".

Utilice el método copy() para crear un nuevo objeto pedidoEnviado cambiando únicamente el estado a "Enviado".

Aplique la desestructuración (componentN()) sobre pedidoEnviado para extraer en variables locales separadas el id, el producto y el estado.

Imprima en consola los datos extraídos.

Ejercicio 8

Crea una clase estándar llamada TrianguloRectangulo que reciba en su constructor primario los dos catetos: base: Double y altura: Double.

Utiliza require() dentro de init para validar que ambos catetos sean mayores que cero.

Agrega una propiedad calculada llamada hipotenusa de tipo Double. Debe utilizar un get() personalizado que calcule y devuelva el valor de la hipotenusa () apoyándote en Math.sqrt() y en la lectura de sus propios atributos mediante this.

Agrega otra propiedad calculada area de tipo Double cuyo get() devuelva .

Ejercicio 9

Diseña una clase llamada Empleado que contenga:

Un constructor primario con nombre: String, puesto: String y salarioBase: Double.

Un bloque init que garantice mediante la función require() que el salarioBase sea estrictamente mayor que 0.0.

Dos constructores secundarios:

El primero solo recibe el nombre y asigna por defecto el puesto "Becario" y un salarioBase de 1000.0.

El segundo recibe el nombre y el puesto, y asigna por defecto un salarioBase de 1800.0.

Ambos constructores secundarios deben delegar correctamente al constructor primario mediante la expresión this().

Ejercicio 10

Crea una clase llamada CuentaBancaria que reciba el titular en el constructor primario y un saldo inicial. La clase debe tener las siguientes restricciones y comportamientos:

1. El saldo no se puede modificar directamente desde fuera de la clase (private set), pero sí se puede leer públicamente.
2. La propiedad pública titular debe devolver siempre el nombre del titular con todas sus letras en mayúsculas mediante un get() personalizado. Si mediante el set() se intenta asignar un nombre vacío o compuesto únicamente por espacios, debe imprimir un mensaje de error sin actualizar el campo de respaldo (field).
3. Define un método ingresar(monto: Double) y un método retirar(monto: Double) que actualicen el saldo de la cuenta validando que los montos sean mayores a cero y que no se retire más saldo del disponible.